

INSO & MAIS



Procesamiento de datos

Miguel Ángel Fernández Díaz
2024-2025



Tema 2:

Spark básico

2. Introducción a Apache Hadoop

Índice

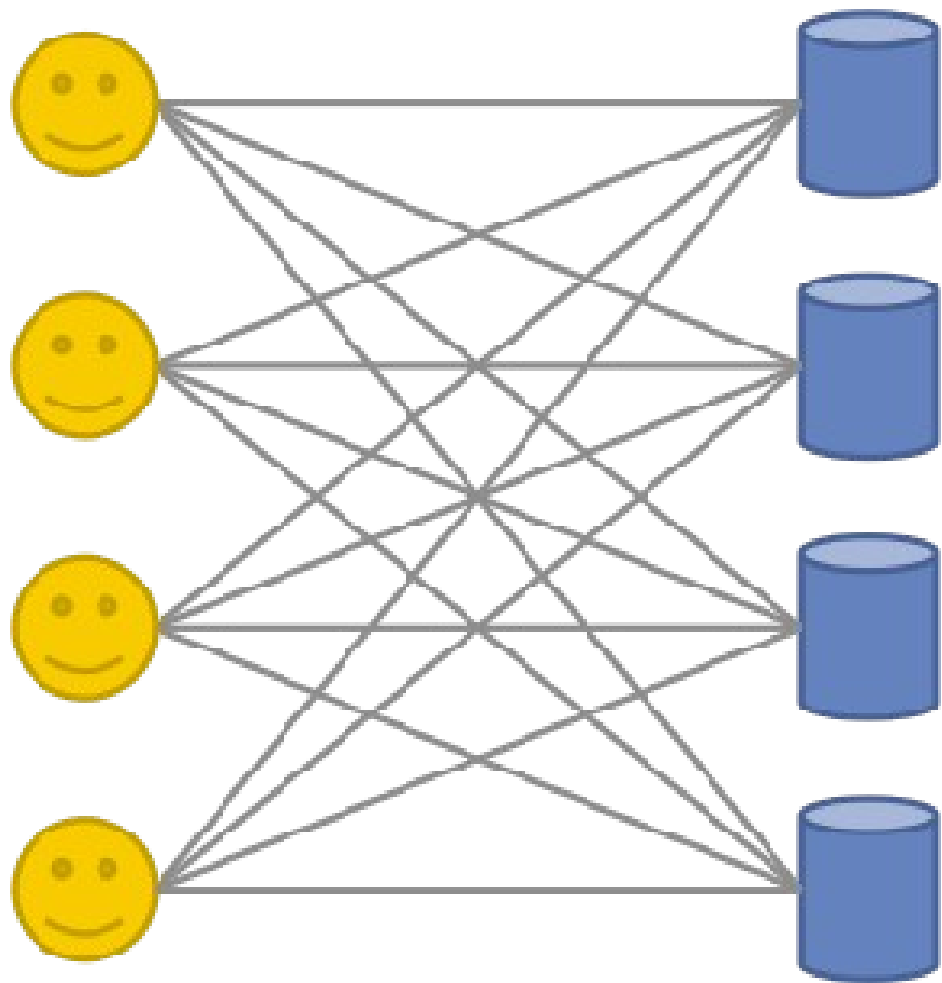
- Historia
- Características
- Componentes
 - HDFS: Almacenamiento de datos
 - MapReduce: Procesamiento de datos
 - YARN: Gestión de recursos

Historia de Apache Hadoop

- Google File System (2003)
- Artículo de Goolge (2004) → MapReduce: Simplified Data Processing on Large Clusters
- Nutch (2005): Motor de búsqueda Web
- Hadoop como solución
- Liberación como proyecto independiente en 2006
 - Yahoo! Adoptó y contribuyó a Hadoop
- En la Apache Foundation desde 2008
- Ecosistema de Hadoop
- Adopción en la industria



Características de Apache Hadoop



Código abierto (Fundación Apache)

Computación distribuida

Tolerancia a fallos

Alta disponibilidad

Extensible

HDFS snapshots

Disk driver awareness

Performance

Escalabilidad

Fiabilidad

Curva de aprendizaje elevada

Localidad del dato



Apoyo de IBM, Oracle, EMC, Cloudera, etc...

Framework para computación distribuida con conjuntos grandes de datos mediante clusters de ordenadores usando programación

Diseñado para escalar de un solo servidor a miles de máquinas que ofrecen tanto capacidad de cómputo como almacenamiento

Diseñado para detectar y tratar fallos de aplicación siendo así un servicio en alta disponibilidad

Diseñado para trabajar con petabytes

Hardware económico (commodity hardware)

Inicialmente, era la única capa de persistencia de otros frameworks Big Data

NO es una base de datos

Modos de despliegue

Local: Modo por defecto que se ejecuta como un único proceso de Java.

- No hay demonios ejecutándose

- El trabajo es ejecutado en la máquina virtual de Java (JVM) local

- El trabajo es ejecutado por el local job runner, una versión más simple del motor de ejecución MapReduce de Hadoop

- Puede ser usado en una fase de desarrollo inicial para depurar el código

Pseudo-distribuido: Varios procesos de java ejecutándose en la misma máquina

- Los demonios de Hadoop se lanzan igual que si fuera un clúster distribuido sólo que todos ellos se ejecutan en la máquina local

- Adecuado para ser usado en la fase de desarrollo ya que permite testar el código en un entorno lo más parecido posible al de producción

Distribuido: Se maximiza el paralelismo de procesos y se utilizan todos los recursos disponibles del clúster en el que se va a configurar Hadoop

- Los demonios de Hadoop se ejecutan en un clúster distribuidos

- Adecuado para producción



Puntos débiles:

- Disco como mecanismo de comunicación entre procesos
- API rudimentaria y con poca riqueza. El concepto MapReduce es funcional, Java no, aunque con matices
- Mucho código para poca funcionalidad
- La cantidad de código repetido es ineficiente.
- Requisitos, en ocasiones, demasiado exigentes.
- Sobreparticionamiento

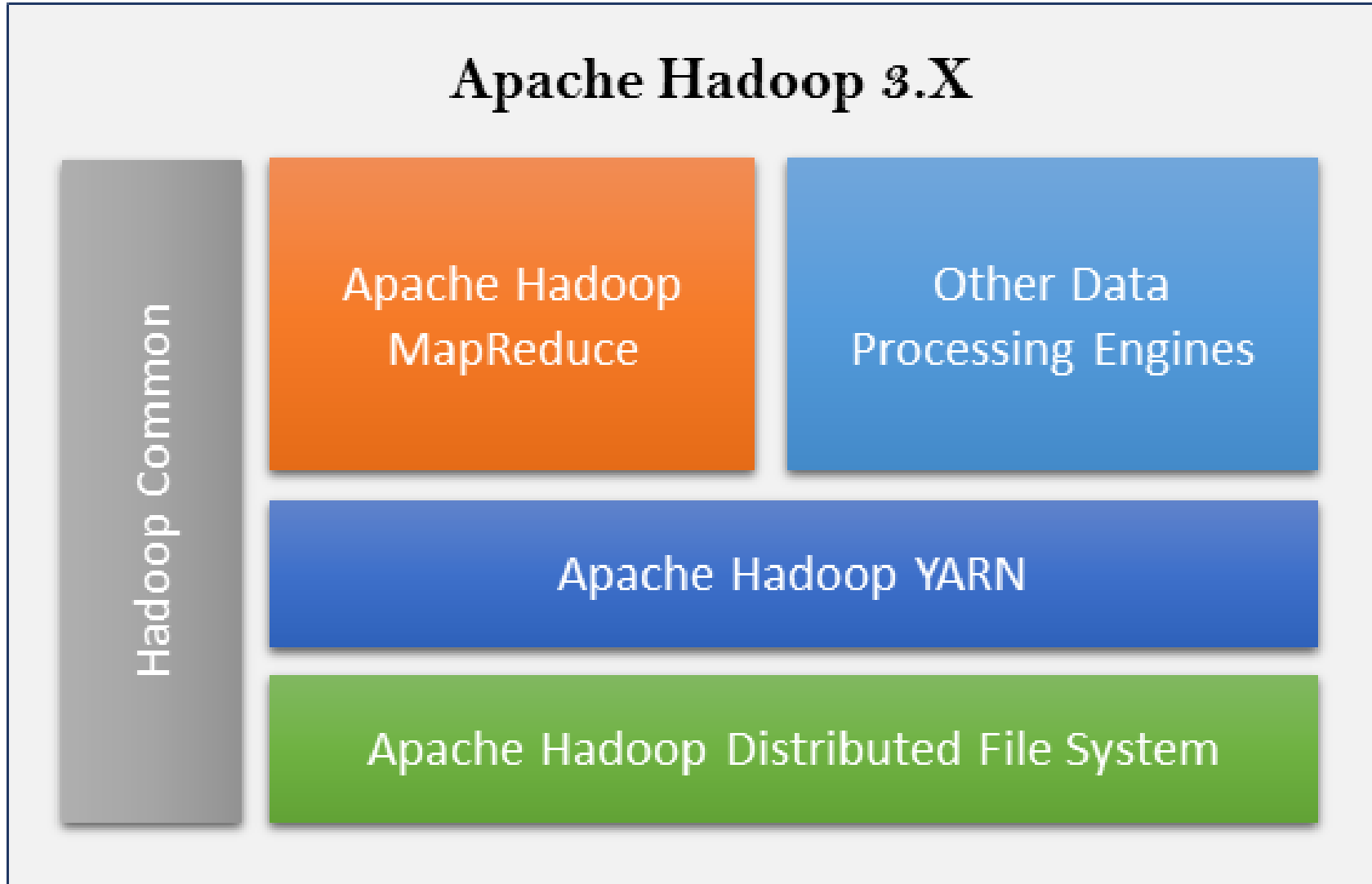


Archivos muy, muy grandes (GB o más)
Necesidad de particionar archivos
Fallo de nodos sin perder información
Una escritura, muchas lecturas



- Alta latencia de red
- Muchos archivos pequeños
- Múltiples “escritores”
- Modificaciones arbitrarias en los archivos

Componentes de Apache Hadoop



HaDooP FileSystem

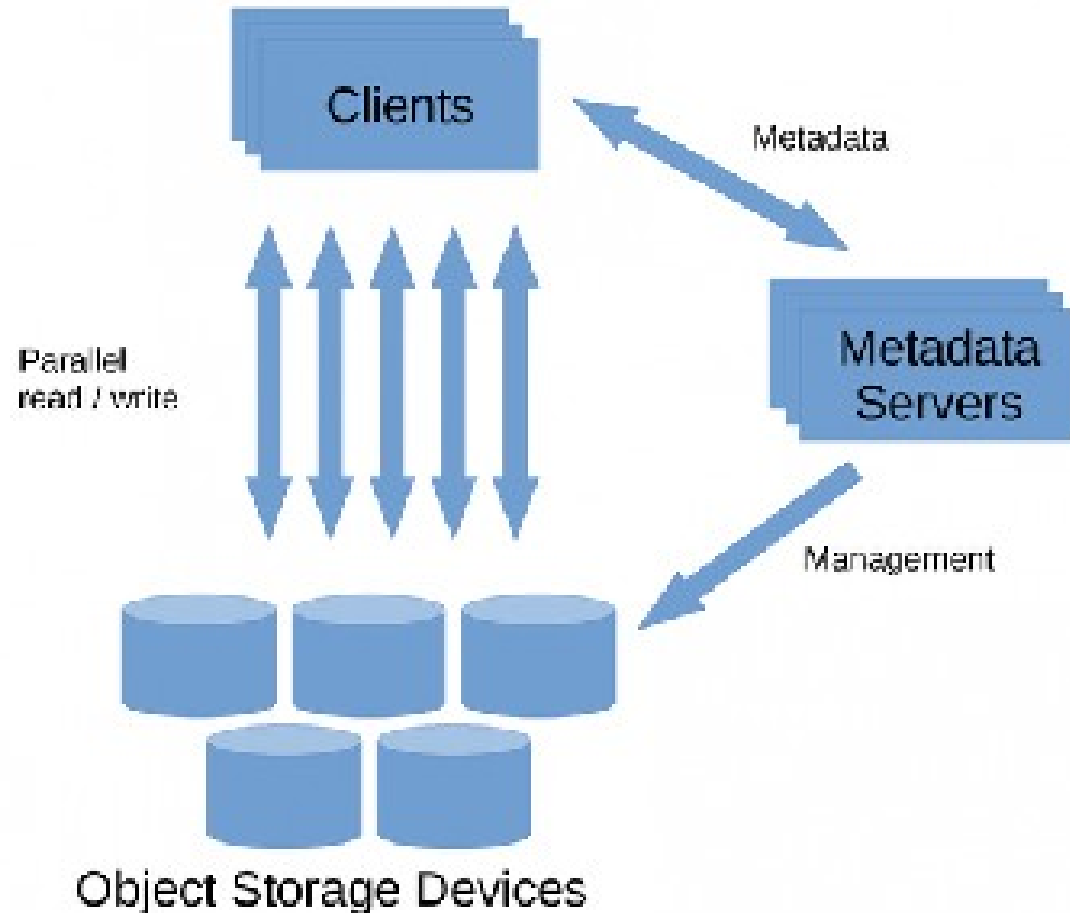
Sistema de ficheros distribuido

Los ficheros que se introducen en el sistema, se trocean y se distribuyen entre los diferentes **datanodes**, que son procesos que escriben y leen datos de una ruta específica y configurada en cada ordenador

El **namenode** conoce qué bloques y dónde están los datos que conforman un fichero

Para un **usuario**, HDFS se ve como un “único disco duro” que contiene ficheros en rutas

El **usuario** utiliza un cliente que se comunica con el **namenode** y los **datanodes**



Inspirado en GFS

Sistema de ficheros distribuido

Estructurado en bloques (típicamente 128MB por bloque)

Rebalanceo de bloques

Ficheros enormes

Tolerancia a fallos

Alta disponibilidad (replicación)

Consistencia

Escalabilidad

Sigue siendo la capa de persistencia más popular para data lakes; es por ello que es muy común utilizarlo sin el resto de componentes de Hadoop

NameNode:

Es la pieza central del HDFS

Administra el almacenamiento de datos

No almacena datos en sí mismo

Las operaciones de Entrada/Salida no pasan a través de él

Hace de intermediario entre el cliente y los DataNodes

Alta disponibilidad: Activo, en espera (stand-by)

- Quorum Journal Nodes

- Almacenamiento compartido

- Failover: Zookeeper

Checkpoint NameNode:

- Copias de seguridad

 - Datos

 - Transacciones del Namenode





DataNode:

- Hasta miles de DataNodes por cluster
- Organizados en racks y datacenters
- Operaciones de Entrada/Salida ocurren sobre el DataNode
- Contienen información replicada
- Alta tolerancia a fallos

Demonios: `start-dfs.sh`

Configuración:

- `core-site.xml`

 - Propiedad: `fs.defaultFS`

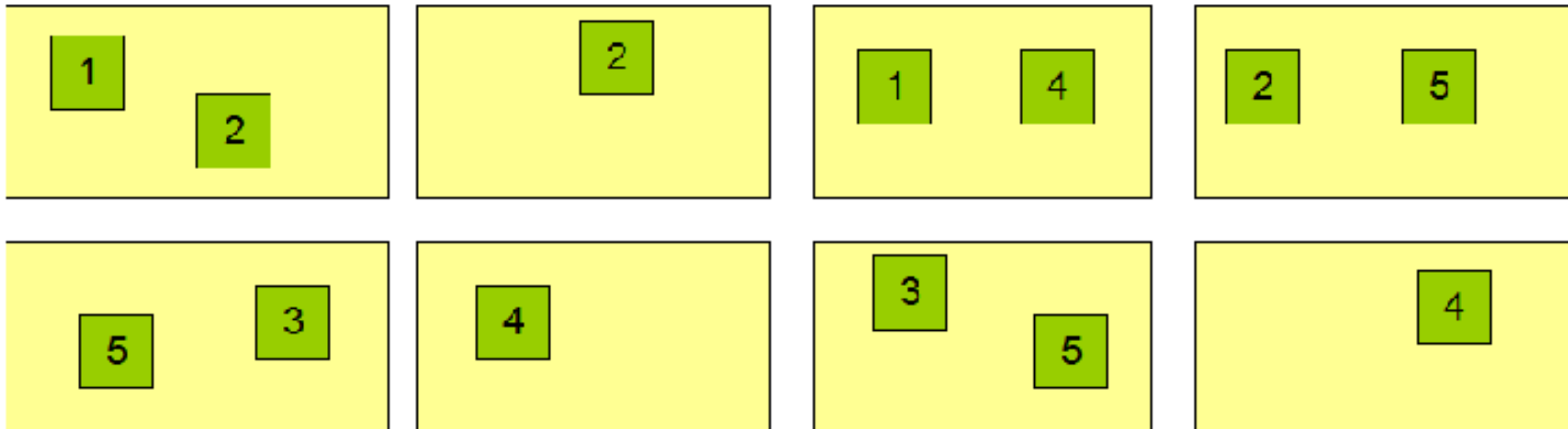
- `hdfs-site.xml`

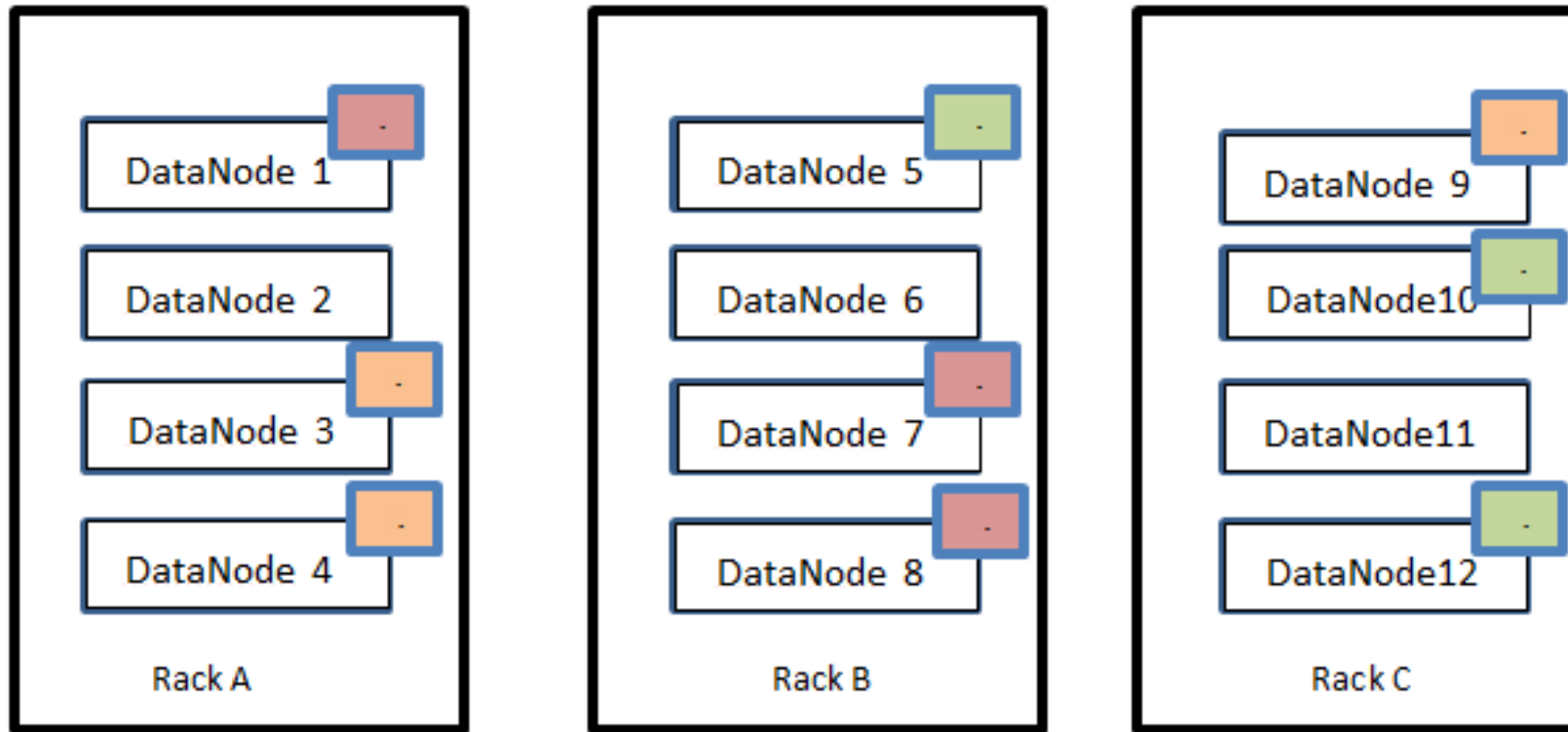
 - `dis.replication`

Block Replication

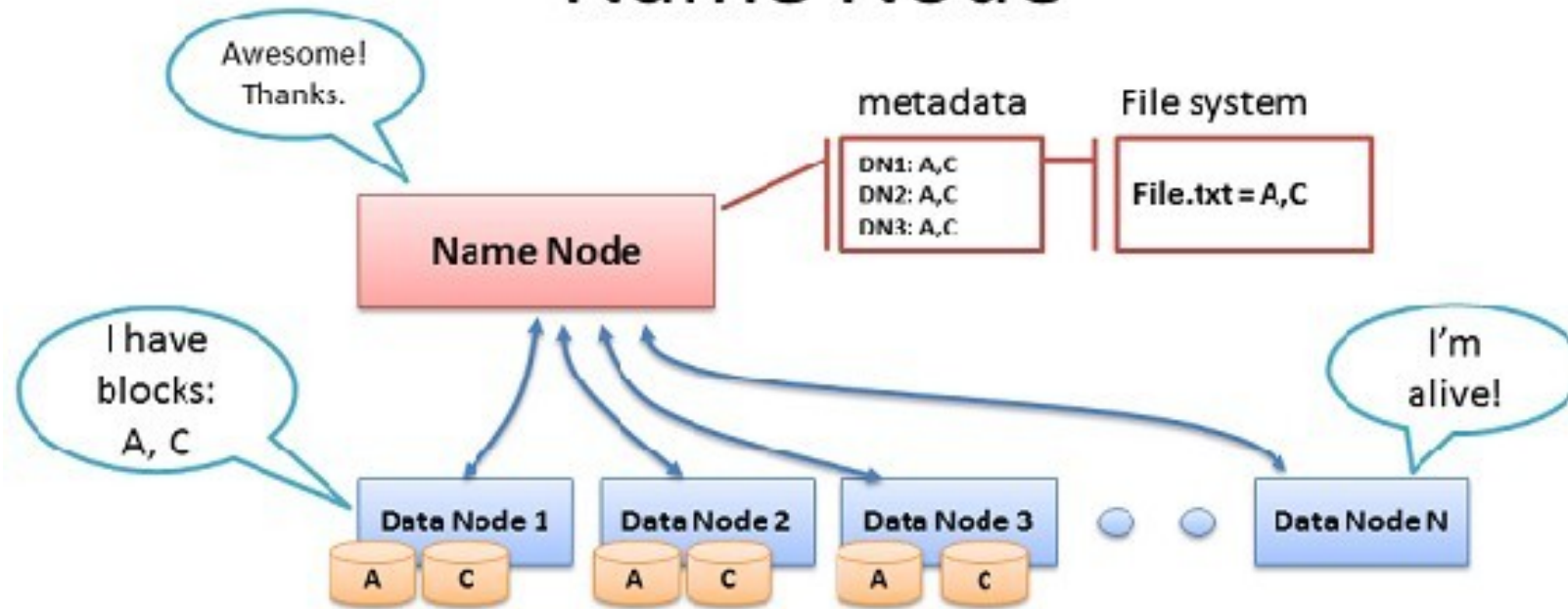
Namenode (Filename, numReplicas, block-ids, ...)
/users/sameerp/data/part-0, r:2, {1,3}, ...
/users/sameerp/data/part-1, r:3, {2,4,5}, ...

Datanodes



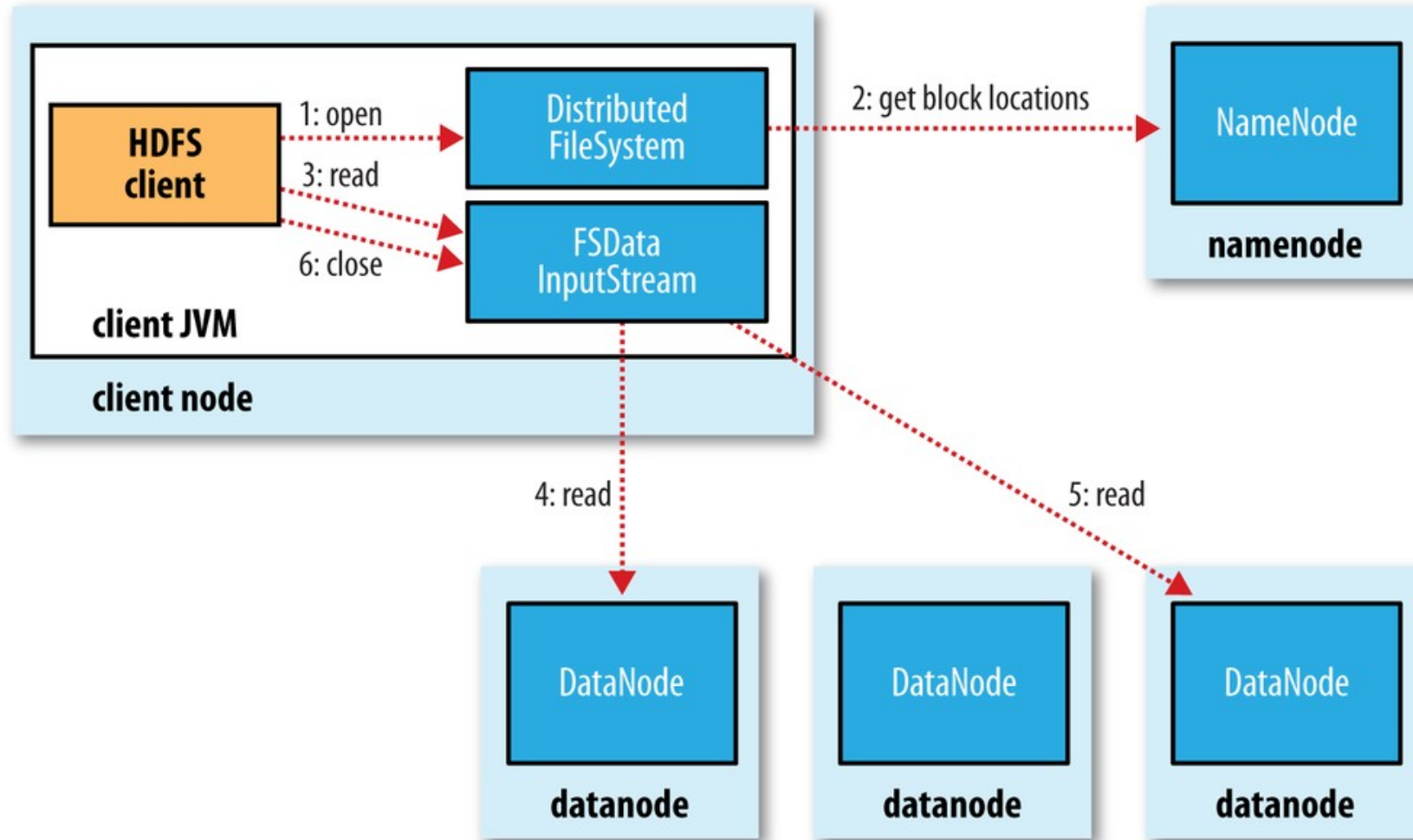


Name Node

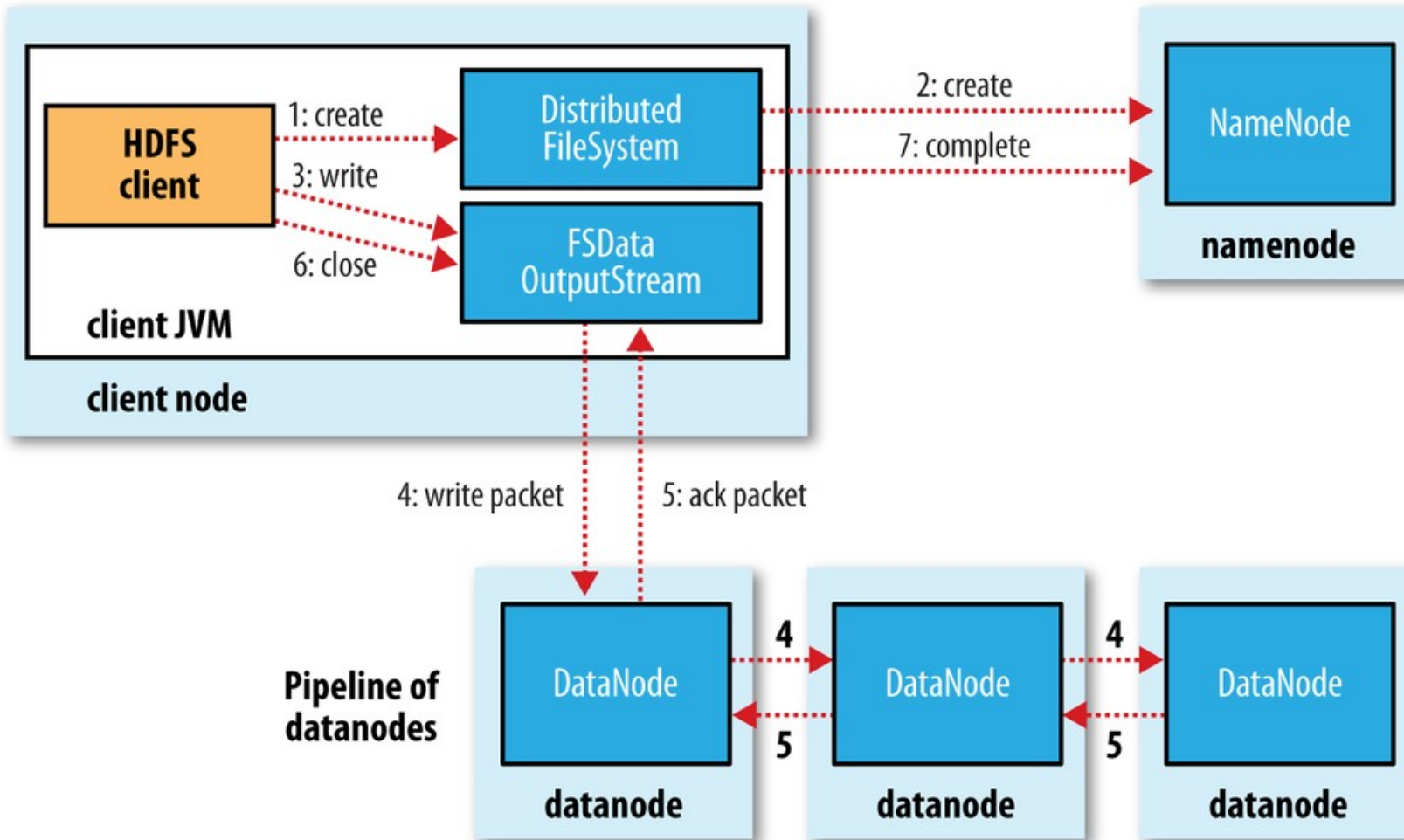


- Data Node sends Heartbeats
- Every 10th heartbeat is a Block report
- Name Node builds metadata from Block reports
- TCP – every 3 seconds
- If Name Node is down, HDFS is down

Componentes de Apache Hadoop: HDFS



Componentes de Apache Hadoop: HDFS



- Formatos:

- Texto
- Fichero Secuencial (clave-valor)
- CSV (Comma Separated Values)
- JSON (JavaScript Object Notation)
- Avro (formato de filas binario compacto con JSON esquema)
- ORC (Optimized Row Columnar)
- Parquet: Formato columnar más popular y más eficiente para filtros de columnas. Menos eficiente para análisis full-scan. Contiene cabecera con información sobre el propio contenido del fichero como número de registros, esquema, etc... Favorece una mejor compresión

```
1 {  
2   "string": "Hi",  
3   "number": 2.5,  
4   "boolean": true,  
5   "null": null,  
6   "object": { "name": "Kyle", "age": 24 },  
7   "array": ["Hello", 5, false, null, { "key": "value", "number": 6 }],  
8   "arrayOfObjects": [  
9     { "name": "Jerry", "age": 28 },  
10    { "name": "Sally", "age": 26 }  
11  ]  
12 }  
13
```

Componentes de Apache Hadoop: MapReduce

Es el componente que permite **analizar** datos almacenados en HDFS

Fase de **Map**: Extrae los datos y los clasifica en forma de una dupla **clave-valor**

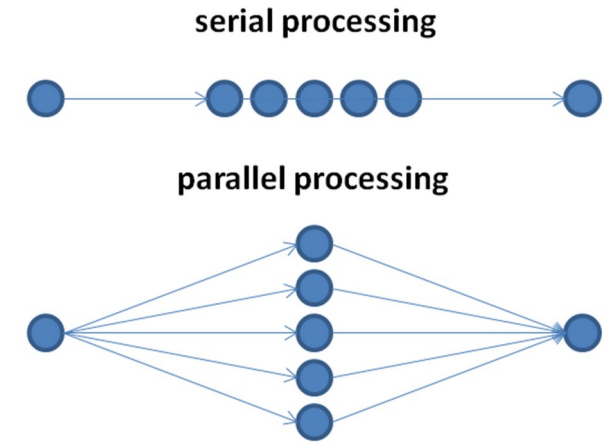
Fase de **Reduce**: Todos los valores de una **misma clave** se recolectan para aplicarles una o varias operaciones

La lógica tanto de la fase de Map como de la fase de Reduce, se implementa a través de la programación de un programa en **Java**.

Hadoop MapReduce se encarga de **paralelizar** ambas fases, es decir, divide las operaciones de cada fase en varias tareas para que varios procesos simultáneos y en diferentes máquinas, puedan realizar el análisis en menos tiempo

Componentes de Apache Hadoop: MapReduce

- Modelo de programación
- Intrínsecamente paralelo
- Fase Map: convertir elementos de un conjunto de datos en un par clave-valor
- Fase Reduce: Combina estos pares para generar un conjunto más reducido de pares
- Creado por Doug Cutting (Universidad de Stanford, Apple, Yahoo, Cloudera)
- Implementación del algoritmo publicado por Google en 2004
- Ejecución de dos procesos separados: Map y Reduce
- Escalabilidad
- Tolerancia a fallos
- Curva de aprendizaje elevada



Componentes de Apache Hadoop: MapReduce



Hadoop proporciona un entorno de ejecución orientado a aplicaciones desarrolladas bajo el modelo de programación MapReduce:

Map: donde se realiza la ingestión y la transformación de los datos de entrada, en la cual los registros de entrada puede ser procesados en paralelo.

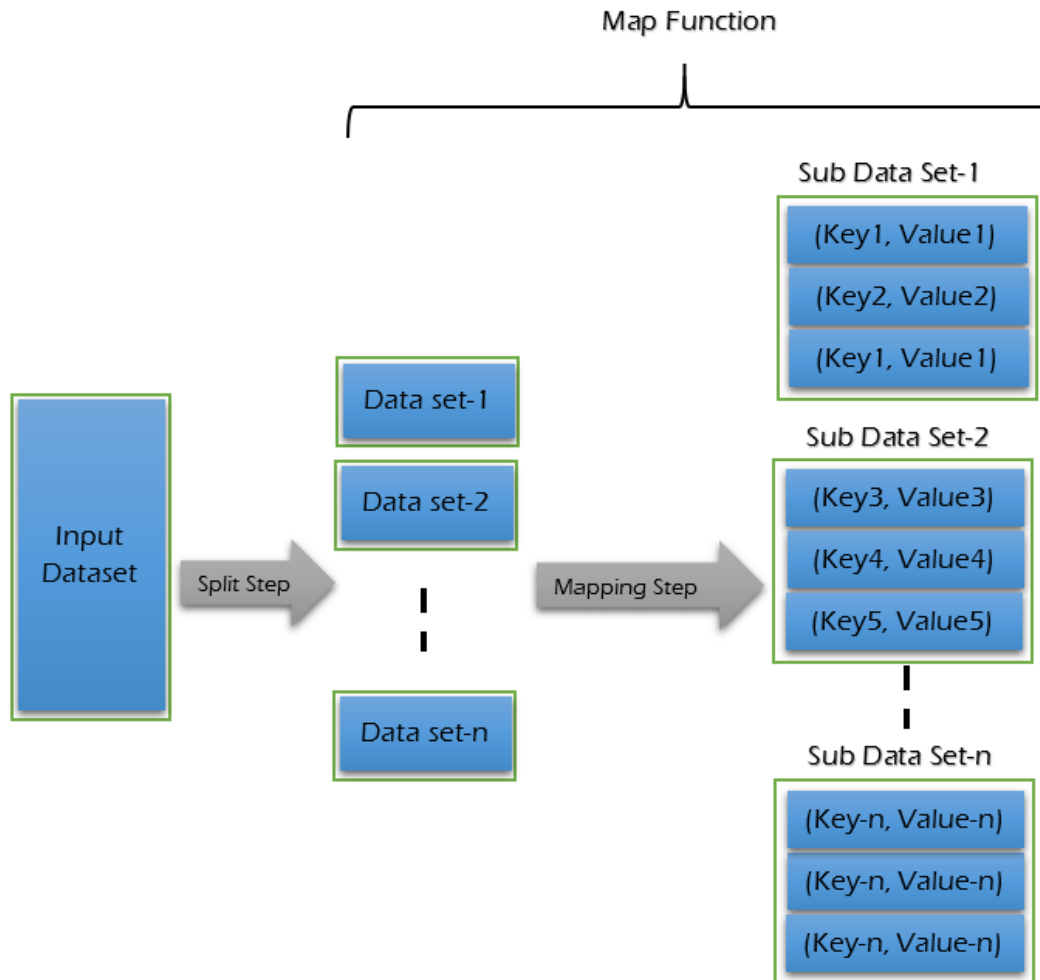
Reduce: fase de agregación donde todos los registros asociados entre sí deben ser procesados juntos por una misma entidad

Demonio: `mr-jobhistory-daemon.sh`

Configuración: `mapred-site.xml`
`mapreduce.framework.name`

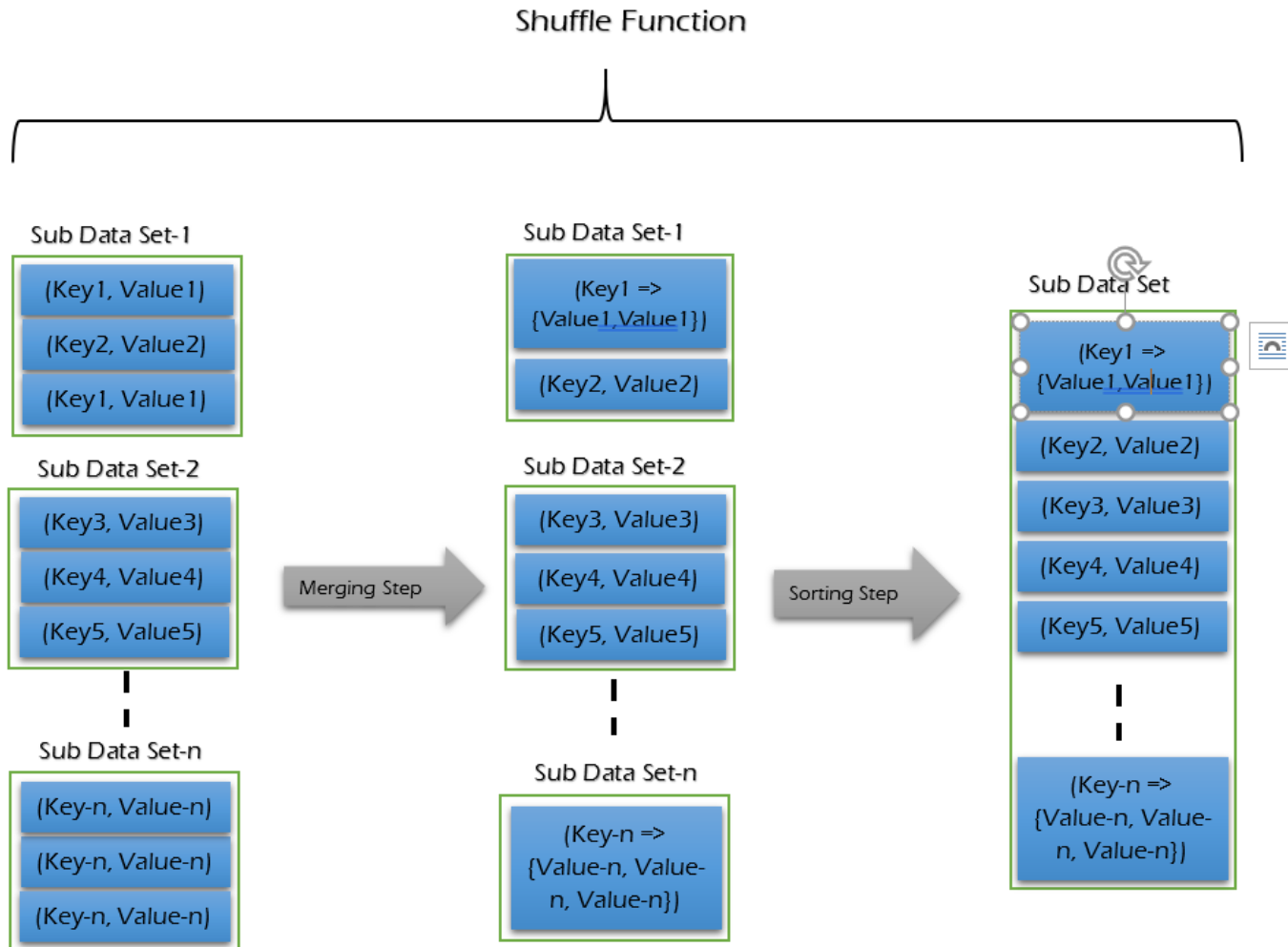
Componentes de Apache Hadoop: MapReduce -tad

Recibe como entrada un conjunto o conjuntos de elementos y los clasifica para devolver como salida tuplas clave-valor



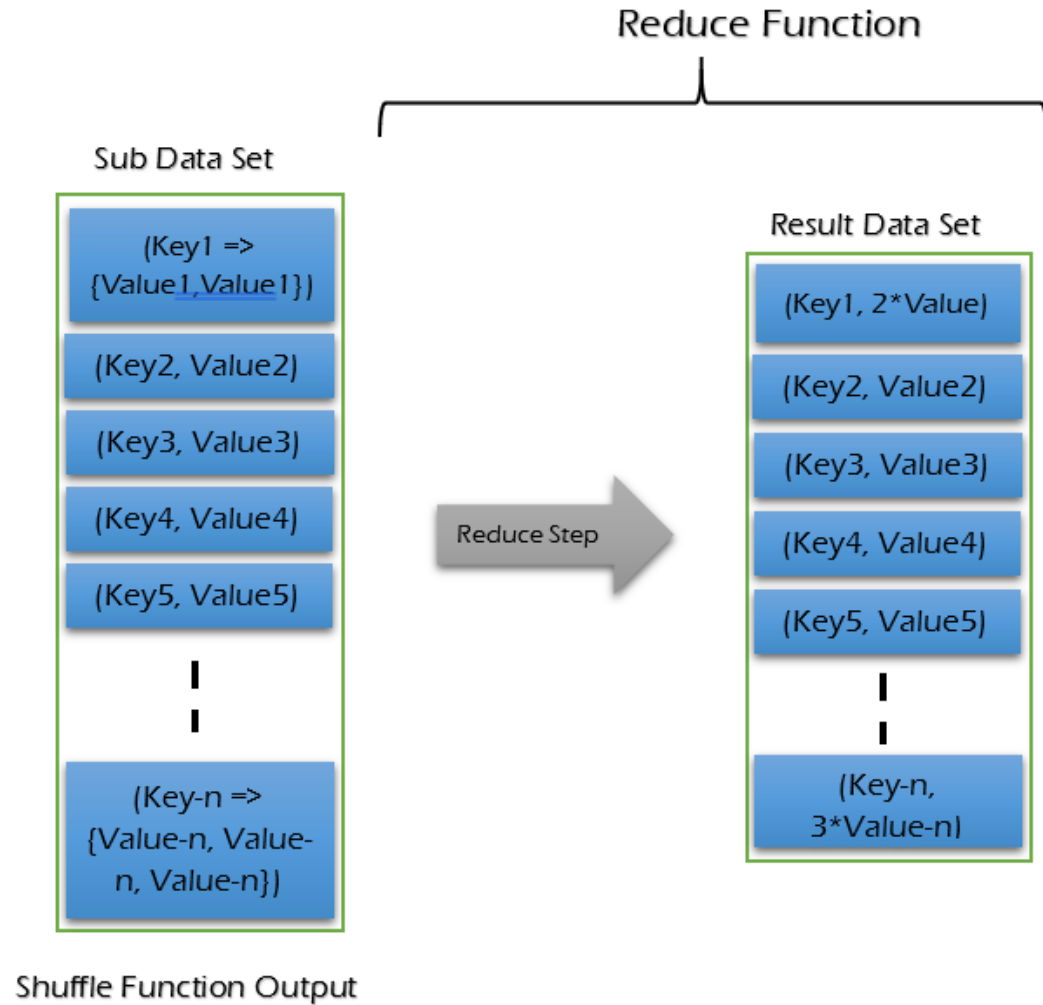
Componentes de Apache Hadoop: MapReduce

Agrupar elementos en función de su clave y los ordena

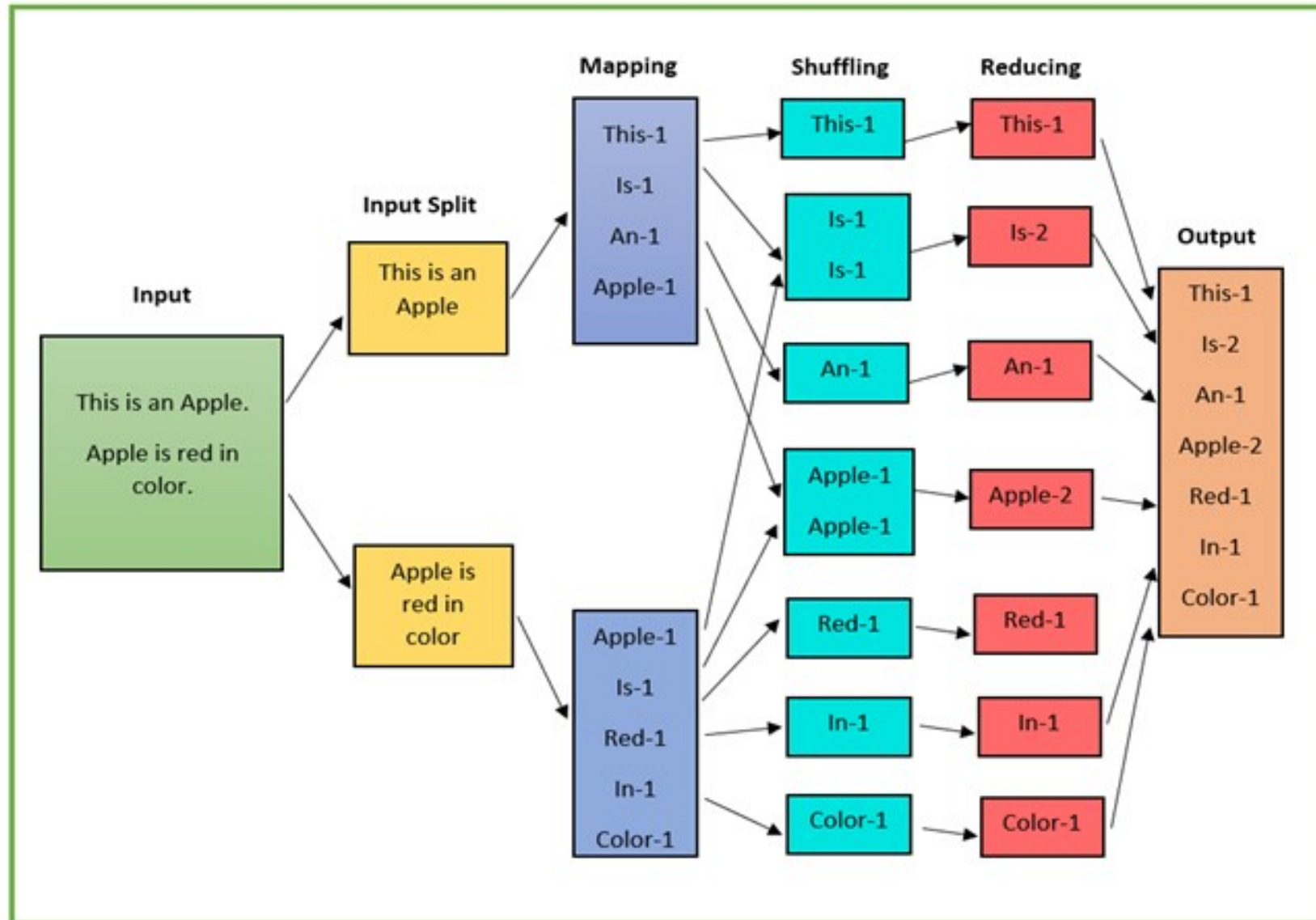


Componentes de Apache Hadoop: MapReduce

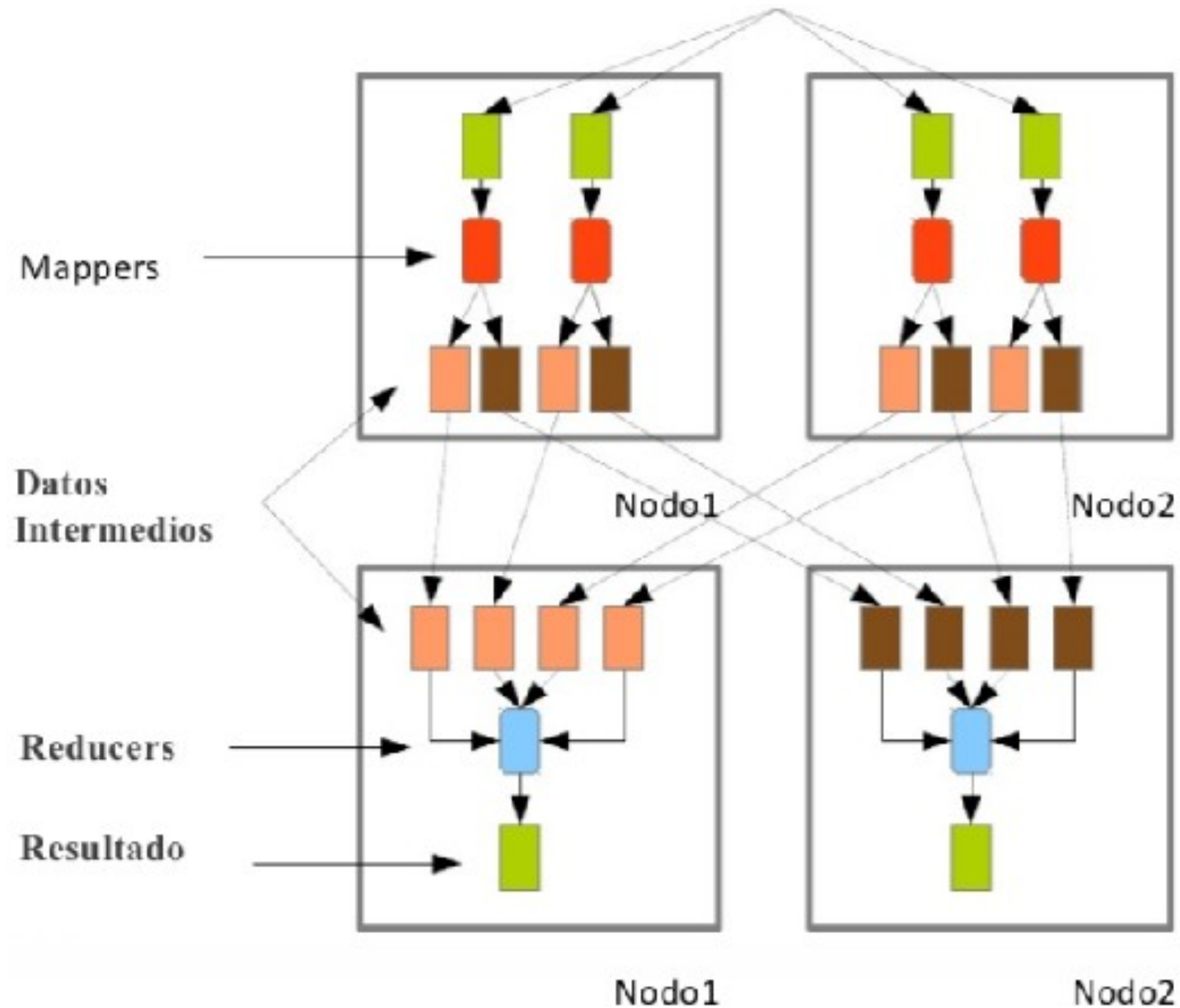
Recibe como entrada un par clave-(lista de valores) y entrega como salida un único par clave-valor



Componentes de Apache Hadoop: MapReduce -Tad



Componentes de Apache Hadoop: MapReduce



Componentes de Apache Hadoop: MapReduce -Tad

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
        ) throws IOException, InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}

public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
        ) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Ejercicio: Calcular la media de pulsaciones de cada equipo de ciclismo cuando sus corredores llegan a meta

33, Contador, Movistar, 110
1, Induráin, Banesto, 87
27, Armstrong, Sky, 99
2, Arroyo, Banesto, 103
9, Olano, Banesto, 119
5, Ocaña, Banesto, 88
22, Smith, Sky, 89
31, Valverde, Movistar, 101
23, Miller, Sky, 98
26, Froome, Sky, 109
34, Quintana, Movistar, 112

Formato:

Dorsal, nombre, equipo, pulsaciones

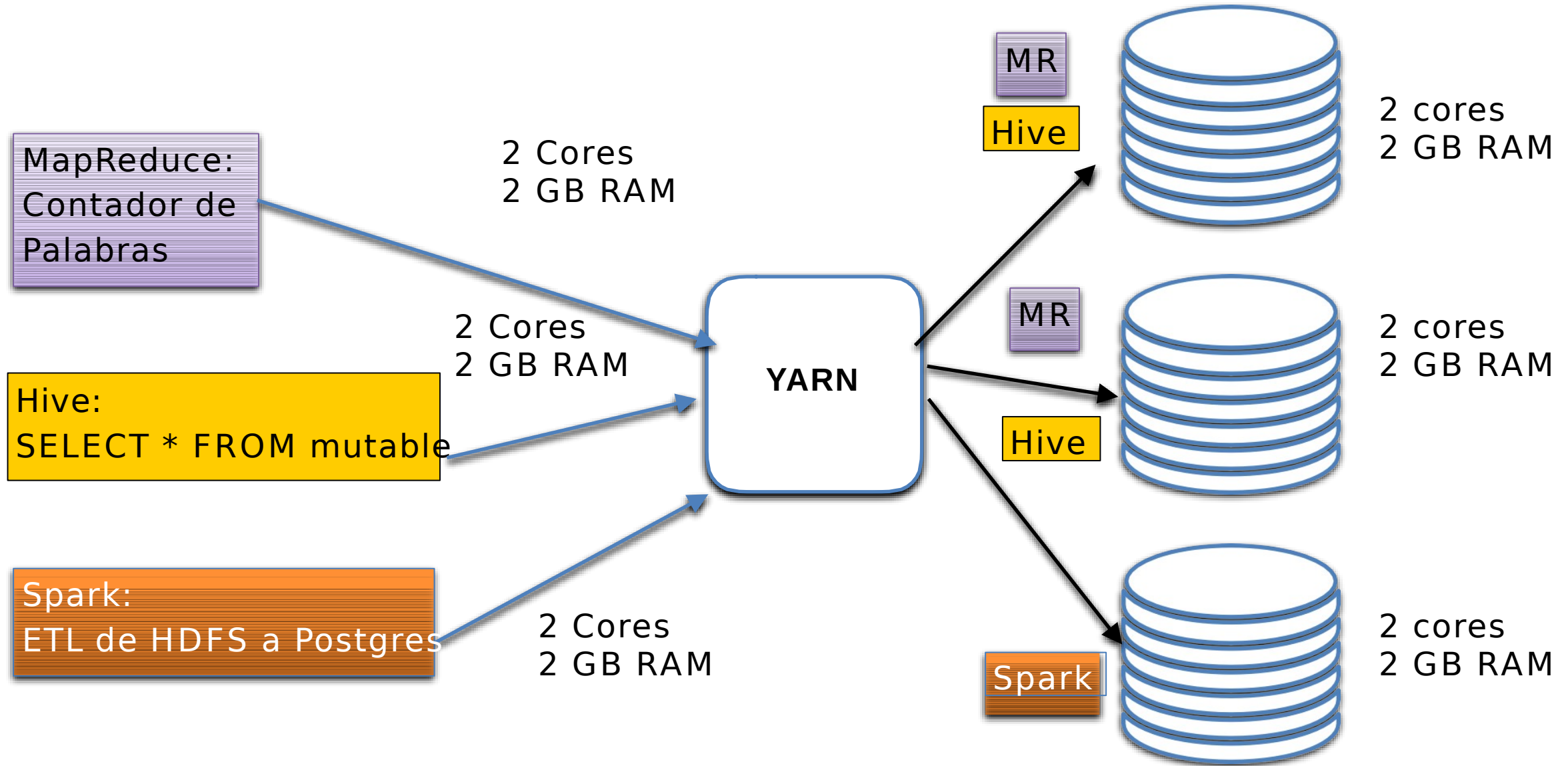
Yet Another Resource Manager

Se encarga de repartir los **recursos** de un cluster entre las diferentes aplicaciones

YARN es una herramienta que se comunica con los nodos del cluster y reserva sus recursos (cores y RAM principalmente) para decidir qué recursos se pueden utilizar para ejecutar cada aplicación.

Una **aplicación** le pide a YARN los recursos que necesita para ejecutarse, YARN mira el listado de recursos “ociosos” y, si hay suficientes recursos libres, le indica a la aplicación qué cores y qué RAM le corresponde de cada máquina.

Componentes de Apache Hadoop: YARN





Sistema operativo distribuido de Hadoop
Yet Another Resource Negotiator

Multi-tenant

Utilización del cluster

- Gestionar recursos

- Planificar tareas

Escalabilidad

Compatibilidad

Optimiza qué nodos deben recibir trabajo

Reasigna trabajos en caso de fallo

Propósito específico: se utiliza para Hadoop o frameworks que utilizan HDFS

Su gran competidor es mesos

Demonio: `start-yarn.conf`

Configuración: `yarn-site.xml`

- `yarn.resourcemanager.hostname`

Tenemos logs del servidor que aloja nuestra página web

Queremos un análisis sobre la actividad de los usuarios en nuestra web

Necesitamos extraer los clicks de cada sección, agrupándolos por hora

- **Formato:** fecha(dia-mes-año-hora:minuto:segundo), usuario, sección, sistema operativo
- Tenemos 3 datanodes (paralelismo)
- Calcula las fases de map, shuffle y reduce

```
Split 1: 17-01-2015-15:21:34, pmadriral, bicicletas, Mac
Split 1: 17-01-2015-15:34:53, darroyo, balones, Windows
Split 1: 17-01-2015-16:19:09, rvasallo, canastas, Android
Split 2: 17-01-2015-16:21:32, darroyo, balones, Windows
Split 2: 17-01-2015-16:35:02, rvasallo, balones, Android
Split 2: 17-01-2015-17:03:52, pmadriral, balones, Mac
Split 3: 17-01-2015-17:22:23, rvasallo, canastas, Android
Split 3: 17-01-2015-17:36:12, darroyo, bicicletas, Windows
Split 3: 17-01-2015-17:39:59, darroyo, bicicletas, Windows
```



 Calle Playa de Liencres, 2 bis
(entrada por calle Rozabella)
Parque Europa Empresarial
Edificio Madrid
28290 Las Rozas, Madrid

 900 373 379  info@u-tad.com

 [SOLICITA MÁS INFORMACIÓN](#)



CENTRO ADSCRITO A:



PROYECTO COFINANCIADO POR:

